

Introduction to JAX

A High-Performance Numerical Computing Library
CSE 521: Embedded Machine Learning

Eng. Omar Tahon

May 19, 2026

Outline

- ➊ Introduction to JAX
- ➋ Just-In-Time Compilation (JIT)
- ➌ Automatic Vectorization (VMAP)
- ➍ Automatic Differentiation (GRAD)
- ➎ Pytrees
- ➏ Conclusion

What is JAX?

JAX is a high-performance numerical computing library developed by Google Research. It brings together three powerful concepts:

- **NumPy API:** JAX provides a familiar `jax.numpy` interface, acting as a drop-in replacement for standard NumPy.
- **Autograd:** It allows for automatic differentiation of native Python and NumPy code.
- **XLA (Accelerated Linear Algebra):** A domain-specific compiler that accelerates math operations across CPUs, GPUs, and TPUs.

Goal: Allow researchers to write simple Python code that scales effortlessly to massive hardware accelerators.

JAX vs. Standard NumPy

While `jax.numpy` mirrors the `numpy` API, it has several superpowers under the hood.

Feature	Standard NumPy	JAX
Hardware Support	CPU only	CPU, GPU, TPU
Execution Model	Eager	Eager & Compiled (JIT)
Derivatives	Manual / Finite Diff	Automatic (<code>grad</code>)
Vectorization	Manual batching	Automatic (<code>vmap</code>)
Mutability	Mutable arrays	Immutable arrays

Table: Comparison of features between NumPy and JAX.

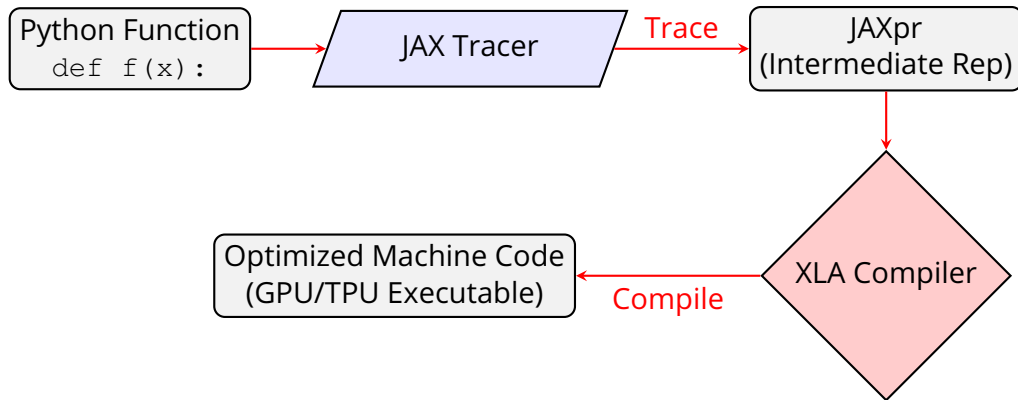
Just-In-Time Compilation (`jax.jit`)

Problem: Standard Python executes line-by-line (eagerly). This introduces significant overhead, especially for complex mathematical functions.

Solution: `jax.jit` traces your Python function and compiles it into highly optimized machine code using the **XLA** compiler.

- **Operation Fusion:** Multiple mathematical operations are merged into a single kernel, avoiding memory read/write bottlenecks.
- **Hardware Optimization:** XLA optimizes the code specifically for the hardware you are running on (GPU/TPU).

Schematic: How `jit` Works



Note: The compilation only happens once (on the first function call). Subsequent calls reuse the compiled executable instantly.

Code Example: `jax.jit`

```
import jax.numpy as jnp
from jax import jit
import time

def slow_selu(x, alpha=1.67, lambda=1.05):
    # Standard Python execution
    return lambda * jnp.where(x > 0, x, alpha * jnp.exp(x) - alpha)

# Compile the function
fast_selu = jit(slow_selu)

x = jnp.arange(1000000.0)
_ = fast_selu(x) # First run compiles (warmup)

# Second run is lightning fast!
res = fast_selu(x)
```

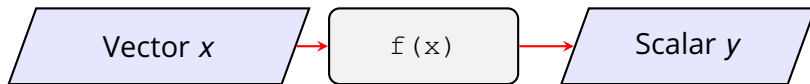
Automatic Vectorization (`jax.vmap`)

Problem: Writing code that handles varying batch sizes usually involves complex, error-prone manual loops or reshaping operations.

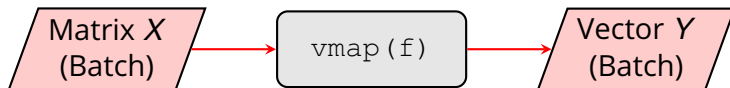
Solution: `jax.vmap` allows you to write a function that operates on a *single* data point. It then automatically transforms this function to operate on a *batch* of data points.

- Keeps code clean and mathematical.
- Pushes the loop down into the C++ / XLA level for performance.

Schematic: How `vmap` Works



Automatic Batching



`vmap` maps the logic across a new dimension without any manual `for` loops.

Code Example: `jax.vmap`

```
from jax import vmap
import jax.numpy as jnp

# Function strictly for 1D vectors
def dot_product(v1, v2):
    return jnp.dot(v1, v2)

# Create two 2D matrices (batches of 10 vectors of size 5)
mat1 = jnp.ones((10, 5))
mat2 = jnp.ones((10, 5))

# Automatically vectorize to handle the matrix inputs
# in_axes=(0, 0) maps over the first axis (rows)
v_dot = vmap(dot_product, in_axes=(0, 0))

result = v_dot(mat1, mat2) # Output shape: (10,)
```

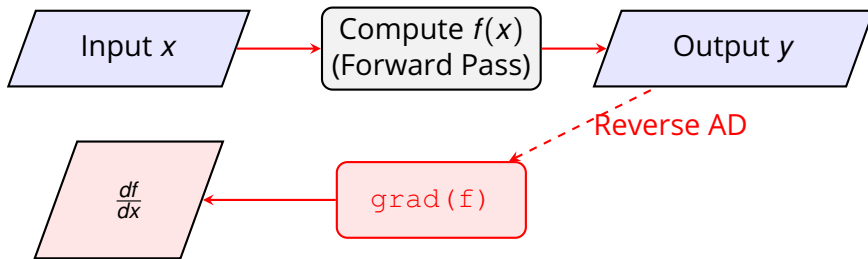
Automatic Differentiation (`jax.grad`)

Problem: Deriving gradients for complex loss functions in Machine Learning by hand is impossible, and finite differences are numerically unstable.

Solution: `jax.grad` provides exact, analytical gradients of scalar-valued functions using Reverse-Mode Automatic Differentiation.

- Automatically computes $\frac{df}{dx}$.
- Can be chained for higher-order derivatives (`grad(grad(f))`).

Schematic: Forward and Backward Pass



`grad` traces the computational graph built during the forward pass and applies the chain rule backward to compute exact derivatives.

Code Example: `jax.grad`

```
from jax import grad

# Define a polynomial:  $f(x) = 3x^2 + 2x + 1$ 
def f(x):
    return 3 * x**2 + 2 * x + 1

# Automatically create the derivative function
# Analytical derivative:  $df/dx = 6x + 2$ 
df_dx = grad(f)

x_val = 2.0
print("f(x)   =", f(x_val))           # Output: 17.0
print("f' (x) =", df_dx(x_val))        # Output: 14.0
```

Working with Pytrees

Problem: Machine learning models have parameters stored in complex, nested dictionaries, lists, and tuples. Iterating through them manually is tedious.

Solution: JAX treats all nested Python structures as **Pytrees**.

- A Pytree is a container of leaf elements.
- JAX provides `jax.tree_map` to apply a function to every leaf in the entire nested structure instantly.

Code Example: Pytrees

```
from jax.tree_util import tree_map

# A complex nested data structure representing model weights
params = {
    'layer1': [1.0, 2.0, 3.0],
    'layer2': {'w': 0.5, 'b': -0.5}
}

# Update all parameters simultaneously without custom loops
# e.g., applying a learning rate update
updated_params = tree_map(lambda x: x - 0.01, params)

print(updated_params)
# {'layer1': [0.99, 1.99, 2.99], 'layer2': {'w': 0.49, 'b': -0.51}}
```

Summary: The Power of Composable Transformations

JAX transforms the way we write mathematical code by allowing arbitrary composition of its core functions:

The Ultimate JAX Pattern

```
fast_batched_gradient_descent = jit(vmap(grad(loss_fn)))
```

- **grad**: Computes the derivative for a single example.
- **vmap**: Vectorizes the gradient computation to process an entire batch.
- **jit**: Compiles the entire batched gradient update into a single, highly optimized GPU/TPU kernel.

Thank You!

Any Questions?